



V.S.B. ENGINEERING COLLEGE

(An Autonomous Institution)

(Anna University Recognized Research Institute)

Approved by AICTE, New Delhi, Affiliated to Anna University, Chennai

NBA Accredited Courses, Accredited by NAAC

MINI PROJECT REPORT

STUDENT ATTENDANCE MANAGEMENT SYSTEM

SUBMITTED BY ,

NAME : Pravina M

REGISTER NUMBER : 922525243160

DEPARTMENT : AIDS

ACADEMIC YEAR : 2026-27

YEAR / SEMESTER : 2nd Year / 3rd Semester

INTRODUCTION

The Student Attendance Management System is a web-based application developed using the Flask framework (Python) and SQLite database. This project is designed to help educational institutions maintain, track, and manage student attendance records efficiently, replacing traditional manual/paper-based attendance registers.

The system allows an administrator to add new student records, view the complete list of students along with their attendance percentage, edit existing records, and delete records that are no longer required. Based on the attendance percentage entered, the system automatically calculates and displays a performance status for each student, making it easy to identify students who need academic attention.

This project demonstrates the practical application of full-stack web development concepts including server-side scripting, database connectivity, dynamic HTML rendering using Jinja2 templates, and CRUD (Create, Read, Update, Delete) operations, all of which are fundamental skills for building real-world data-driven applications.

KEY FEATURES

- Add new student records with Name, Register Number, Department, and Attendance percentage.
- View all student records in a clean, tabular home page layout.
- Edit and update existing student details at any time.
- Delete student records that are no longer needed.
- Automatic performance status calculation - Excellent, Good, or Needs Improvement - based on attendance percentage.
- Lightweight SQLite database that requires no separate database server setup.
- Simple and responsive user interface built with HTML templates rendered through Flask (Jinja2).
- Persistent storage - all records remain saved even after the application is closed and reopened.

MODULES

The project is broadly divided into the following functional modules:

1. Home / View Module

Displays all student records fetched from the database in a table, showing ID, Name, Register Number, Department, Attendance, and Performance Status.

2. Add Student Module

Provides a form to enter a new student's details. On submission, the performance status is calculated and the record is inserted into the database.

3. Edit Student Module

Fetches the existing details of a selected student, allows the admin to modify them, and updates the record in the database.

4. Delete Student Module

Removes a selected student's record permanently from the database based on the record ID.

5. Database Module

Manages the SQLite connection and defines the 'student' table schema used to persist all records.

SYSTEM ARCHITECTURE

The system follows a simple three-tier architecture consisting of the Client (browser), the Flask application server, and the SQLite database. The browser sends HTTP requests to Flask routes, Flask processes the logic and queries the database, and the resulting data is rendered back to the browser as HTML using Jinja2 templates.

System Architecture - Student Attendance Management System

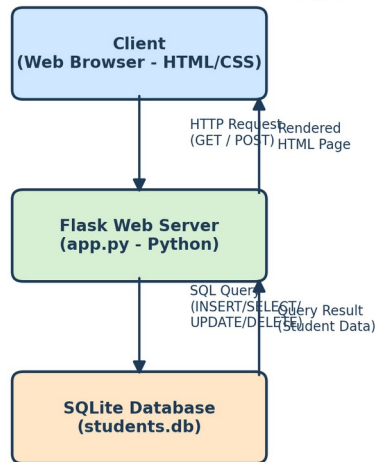


Fig 5.1 - System Architecture of the Student Attendance Management System

IMPLEMENTATION (CODING)

The backend of the application is implemented in Python using the Flask micro-framework. The core file, app.py, defines the database connection, the routes for each module, and the logic to calculate student performance status. The complete code is given below:

app.py

```
from flask import Flask, render_template, request, redirect
import sqlite3

app = Flask(__name__)

# Database Connection
def connect():
    conn = sqlite3.connect("students.db")
    conn.execute("""
    CREATE TABLE IF NOT EXISTS student(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT,
        regno TEXT,
        department TEXT,
        attendance INTEGER,
        status TEXT
    )
    """)
    return conn

# Home Page
@app.route("/")
def index():
    conn = connect()
    data = conn.execute("SELECT * FROM student").fetchall()
    conn.close()
    return render_template("index.html", students=data)

# Add Student
@app.route("/add", methods=["GET", "POST"])
def add():
    if request.method == "POST":
        name = request.form["name"]
        regno = request.form["regno"]
        department = request.form["department"]
        attendance = int(request.form["attendance"])

        # Performance Status
        if attendance >= 90:
            status = "Excellent"
        elif attendance >= 75:
            status = "Good"
        else:
            status = "Needs Improvement"

        conn = connect()

        conn.execute("""
        INSERT INTO student(name, regno, department, attendance, status)
        VALUES (?, ?, ?, ?, ?)
        """, (name, regno, department, attendance, status))

        conn.commit()
        conn.close()

        return redirect("/")
```

```

        return render_template("add.html")

# Delete Student
@app.route("/delete/<int:id>")
def delete(id):
    conn = connect()
    conn.execute("DELETE FROM student WHERE id=?", (id,))
    conn.commit()
    conn.close()
    return redirect("/")

# Edit Student
@app.route("/edit/<int:id>", methods=["GET", "POST"])
def edit(id):

    conn = connect()

    if request.method == "POST":

        name = request.form["name"]
        regno = request.form["regno"]
        department = request.form["department"]
        attendance = int(request.form["attendance"])

        if attendance >= 90:
            status = "Excellent"
        elif attendance >= 75:
            status = "Good"
        else:
            status = "Needs Improvement"

        conn.execute("""
        UPDATE student
        SET name=?, regno=?, department=?, attendance=?, status=?
        WHERE id=?
        """, (name, regno, department, attendance, status, id))

        conn.commit()
        conn.close()

        return redirect("/")

    student = conn.execute(
        "SELECT * FROM student WHERE id=?",
        (id,)
    ).fetchone()

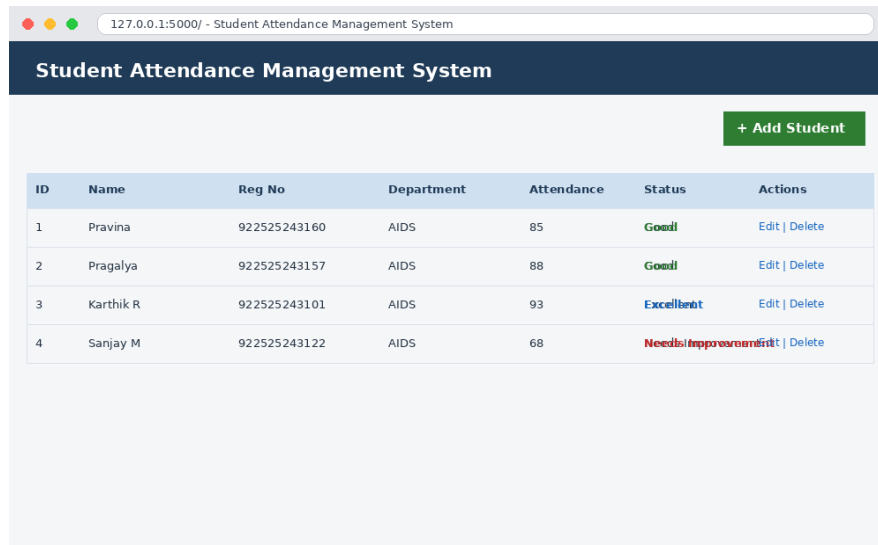
    conn.close()

    return render_template("edit.html", student=student)

if __name__ == "__main__":
    app.run(debug=True)

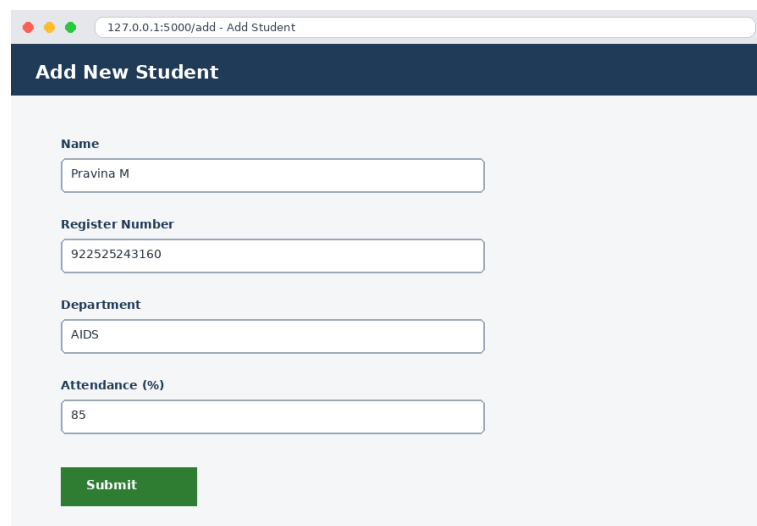
```

OUTPUT SCREENSHOTS



ID	Name	Reg No	Department	Attendance	Status	Actions
1	Pravina	922525243160	AIDS	85	Good	Edit Delete
2	Pragalya	922525243157	AIDS	88	Good	Edit Delete
3	Karthik R	922525243101	AIDS	93	Excellent	Edit Delete
4	Sanjay M	922525243122	AIDS	68	Needs Improvement	Edit Delete

Fig 7.1 - Home Page displaying the list of students



Add New Student

Name

Register Number

Department

Attendance (%)

Fig 7.2 - Add Student form

127.0.0.1:5000/edit/1 - Edit Student

Edit Student Details

Name
Pravina M

Register Number
922525243160

Department
AIDS

Attendance (%)
90

Update

Fig 7.3 - Edit Student form

DIAGRAMS

Entity-Relationship Diagram - Student Table

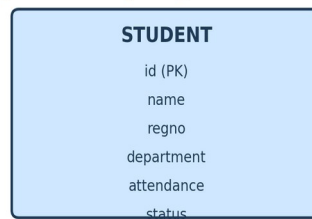


Fig 8.1 - Entity Relationship (ER) Diagram of the Student table

Application Workflow Diagram

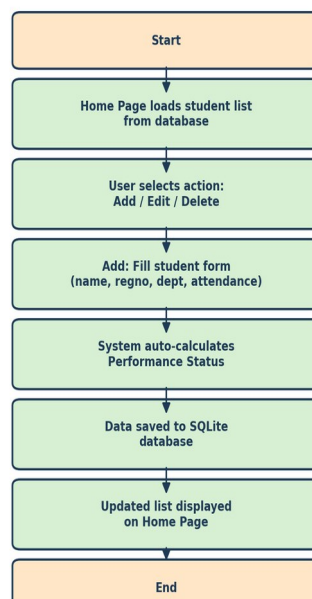


Fig 8.2 - Application Workflow Diagram

SOFTWARES USED

Category	Software / Tool
Operating System	Windows 10 / 11
Programming Language	Python 3
Web Framework	Flask
Database	SQLite
Code Editor / IDE	Visual Studio Code
Browser	Google Chrome
Template Engine	Jinja2 (HTML)

TECHNOLOGY STACK

Frontend:

- HTML5 - Page structure
- CSS3 - Styling of pages
- Jinja2 - Dynamic template rendering

Backend:

- Python 3
- Flask Web Framework

Database:

- SQLite3

ADVANTAGES AND DISADVANTAGES

Advantages

- Eliminates manual, paper-based attendance registers.
- Quick and easy to add, update, or delete student records.
- Automatic performance status calculation saves manual effort.
- Lightweight database (SQLite) requires no separate installation.
- Simple, easy-to-use interface suitable for non-technical staff.

Disadvantages

- No login/authentication system, so anyone with access can modify records.
- SQLite is not ideal for handling very large volumes of concurrent users.
- No data validation for duplicate register numbers.
- Lacks features like attendance history tracking date-wise or report export (PDF/Excel).

CONCLUSION

The Student Attendance Management System successfully demonstrates how a simple, efficient web application can be built using Flask and SQLite to manage student attendance records. The project covers the complete CRUD cycle - adding, viewing, editing, and deleting records - along with automatic performance evaluation based on attendance percentage.

This mini project strengthened the understanding of full-stack web development, including backend routing, database integration, and dynamic HTML rendering. In the future, the system can be enhanced with login authentication, date-wise attendance tracking, and report export features to make it suitable for real-world deployment in educational institutions.

REFERENCES

- Flask Official Documentation - <https://flask.palletsprojects.com/>
- Python Official Documentation - <https://docs.python.org/3/>
- SQLite Official Documentation - <https://www.sqlite.org/docs.html>
- Jinja2 Template Documentation - <https://jinja.palletsprojects.com/>
- W3Schools - HTML/CSS Reference - <https://www.w3schools.com/>

PROJECT / GITHUB LINK

GitHub Repository Link:

[Add your GitHub repository link here after uploading the project]

THANK YOU !!